

Verification Methodology · Test Automation · 2026

이시영 — SW 검증 자동화 포트폴리오

게임 시스템을 대상으로 한 자동화 프로젝트로
상태 기반 검증 · 이상치 판정 · 반복 회귀 방법론을 실습했고,
현대오토에버 Embedded SWQA 관점에서 정리했습니다.

대상 직무
Embedded SWQA

검증 방법론 실습
게임 시스템 자동화
FSM · POM · Sanity check

기술 스택
Python · Git · OCR · OpenCV

게임을 대상으로 한 검증 자동화 방법론 실습

이 프로젝트는 무엇인가

자동차 임베디드 경험 대신, 검증 자동화의 사고 프레임워크를 게임을 대상으로 실습했습니다.

반복 검증에서 사람의 주의력은 빠르게 떨어지고 오탐·미탐이 발생할 가능성이 높아집니다. 사람이 눈으로 매번 확인하던 6개 메커니즘을 자동화하면서 문제를 직접 부딪혔습니다.

- ① 상태가 변하는 시스템을 어떻게 모델링할 것인가
- ② 받은 신호(OCR)를 그대로 믿을 것인가
- ③ 재현 가능한 증거를 어떻게 남길 것인가

→ 이 세 가지는 Embedded SWQA에서도 동일하게 마주치는 문제입니다.

자동화한 6개 메커니즘

검증 항목	수동 한계	자동화 해결
발판 이동 정상 동작	발판 위치를 매번 기억해야 함	HSV + 템플릿 매칭
전투 진입 확인	화면 전환 타이밍에 따라 놓칠 수 있음	AUTO 버튼 감지로 진입 여부 판단
★ EX 스킬 사용 감지	전투 중 눈으로 추적 불가	코스트 변화 + Canny 엣지 실시간 감지
코스트 소모 검증	숫자를 빠르게 읽기 어려움	OCR로 스킬 전후 코스트 자동 기록
데미지 수치 기록	결과 화면을 빠르게 넘겨버림	리포트 팝업 OCR → JSON 저장
보상 획득 확인	아이템 목록 수동 기록	보상 팝업 OCR → JSON 저장

SWQA Outcome

6개 메커니즘 × 무인 반복 수행 → test_summary.json 한 파일에 PASS/FAIL 자동 집계

Every stage produced a real artifact

V-model 단계별 산출물 정리

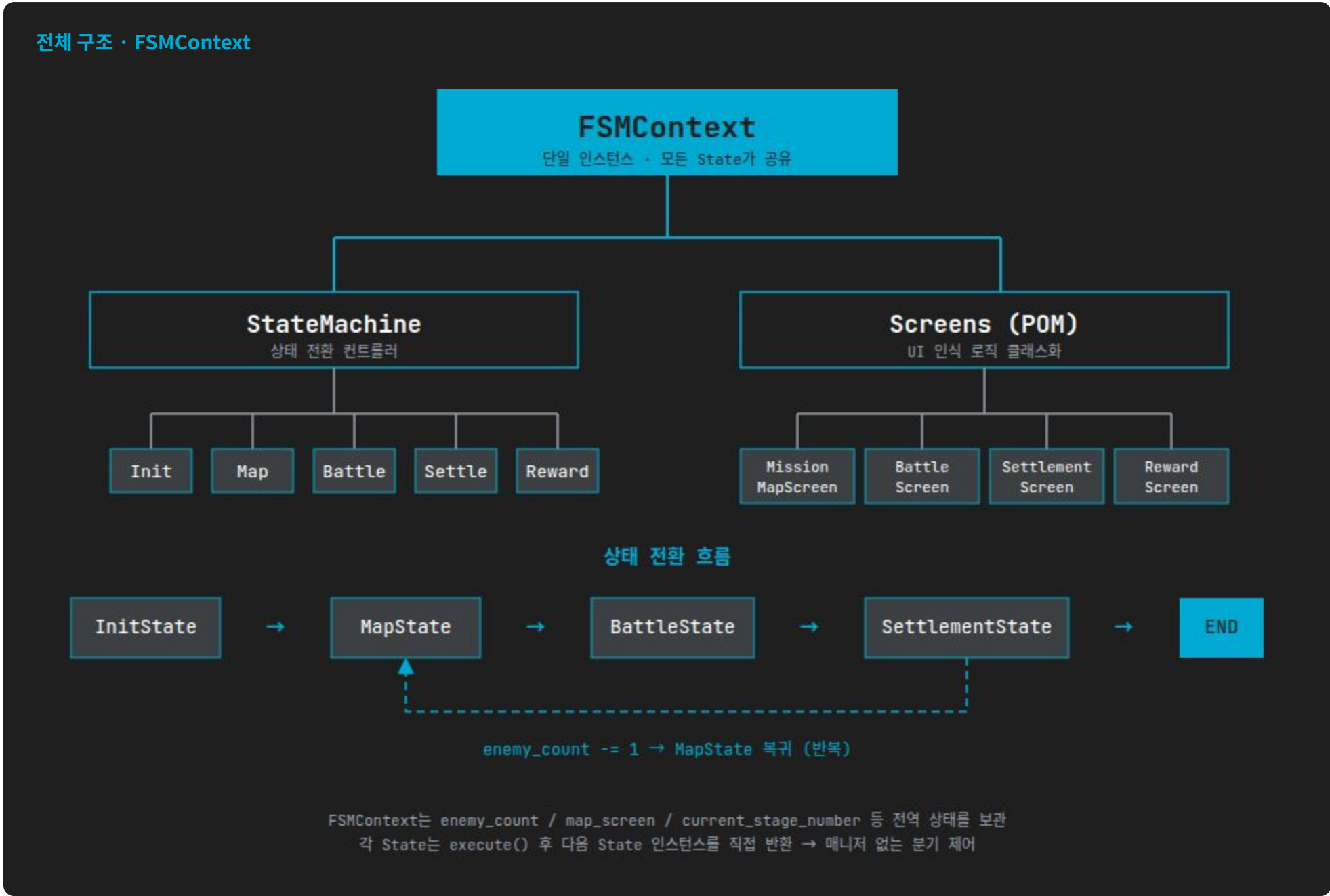


Embedded SWQA 실무 항목 · V-model 대응

- › 요구사항 분석 · Trace matrix 작성 → Stage 01 — 6개 검증 항목의 근거를 게임 메커니즘 명세와 1:1 매핑 (실제 embedded에서는 SW 요구사항 ↔ TC ID)
- › TC 설계 · Boundary / Equivalence 파티셔닝 → Stage 02 — 코스트 값 범위 (0~10) 기반 경계값 TC 설계, sanity check 규칙으로 반영
- › 단위 · 통합 테스트 자동화 (Google Test 등) → Stage 03 — Screen / State 클래스 단위로 개별 검증 후 FSM 통합 (Python 기반 실습)
- › HIL 반복 회귀 · 이슈 리포트 → Stage 04·05 — 30회 무인 반복 실행 결과를 재현 가능한 JSON 패키지로 산출

Three patterns combined

아키텍처 — FSM + POM + Singleton



Pattern 01

FSM — 상태 머신

각 단계를 독립 클래스로 분리해 `if-else` 중첩을 제거했습니다. 모든 State는 BaseState(ABC)를 상속해 `enter` → `execute` → `exit` 생명주기를 강제 — `@abstractmethod`로 구현 누락을 런타임 차단합니다.

Pattern 02

POM — Page Object Model

각 게임 화면의 인식 로직을 Screen 클래스로 캡슐화. UI가 바뀌어도 해당 Screen만 수정. **Selenium / Appium**의 **PageObject**와 동일 원리 — SWQA에서도 즉시 활용 가능.

Pattern 03

Singleton — FileManager

로그·스크린샷 디렉터리 접근을 단일 인스턴스로 통제. 멀티 인스턴스가 같은 파일을 동시에 쓰며 깨지는 사고 차단. `__new__` 오버라이드로 구현.

Detect skill → log event → transition

BattleState — 핵심 흐름만

states/battle_state.py · execute() 핵심 발췌

```
def execute(self) -> BaseState:
    screenshot = capture_window_image(self.context.hwnd)

    # 첫 진입은 MapState에서 이미 확인 → 즉시 모니터링 시작
    is_battle = True if self.first_run else \
        self.context.battle_screen.is_in_battle(screenshot)
    self.first_run = False

    if is_battle:
        # ★ 스킬 감지 핵심 로직
        event = self.context.battle_screen.update_monitor(screenshot)
    else:
        event = self.context.battle_screen._check_battle_end(screenshot, datetime.now())

    if event and event.get("type") == "BATTLE_END":
        # 스킬 사용 기록 FSMContext에 누적
        for skill_event in self.context.battle_screen.events:
            self.context.test_results["skill_uses"].append({
                "student": skill_event.get("student_name"),
                "slot": skill_event.get("slot"),
                "cost": skill_event.get("cost_diff"),
            })
        return SettlementState(self.context)
```

설계 포인트

① POM 위임 — State는 "무엇을"만

update_monitor() 한 줄 호출로 화면 분석 로직을 Screen에 위임. UI 변경 시 State 코드는 수정하지 않습니다.

② 이벤트 객체로 의사소통

Screen은 SKILL_USED 타입 이벤트를 반환. State는 type만 보고 분기 — 결합도가 낮아 단위 테스트가 쉽습니다.

③ 다음 상태 객체 직접 반환

매니저 없이 분기 안전. StateMachine은 `isinstance` 한 줄로만 검사하면 됩니다.

전투 화면에서 스킬 사용 후 로그

```
[18:55:50] [INFO] [BATTLE] 전투 진입 직후 즉시 모니터링 시작
[18:55:51] [INFO] [BATTLE] 게이지 변화: 0 → 3 (↑3)
[18:55:52] [INFO] [BATTLE] 게이지 변화: 3 → 4 (↑1)
[18:55:54] [INFO] [BATTLE] 게이지 변화: 4 → 0 (↓4)
[18:55:54] [INFO] [BATTLE] 스킬 사용 : [후우카] SKILL_1 (소모: 5) ✓
```



assets/screenshots/09-battle-skill-log.png

Trigger → identify → evidence

스킬 감지 — 3단계 이중 검증



트레이드오프 — 오탐 vs 미탐

쿨다운 0.7초 — 안전 마진 확보

스킬의 이펙트가 강해서 UI 깜빡임을 스킬로 오인식이 되는 경우가 있습니다. 한번 스킬 사용을 감지하면 0.7초의 쿨타임을 줘서 스킬 사용 감지를 오탐지 하는 것을 방지합니다.

QA 결정: 안정성 우선으로 오탐 차단. 미탐은 BEFORE/AFTER 비교로 사후 회수.

스킬 사용 BEFORE / AFTER 비교 캡처

SKILL_*_BEFORE.png / AFTER.png



Crop → HSV → Resize → EasyOCR + Sanity check

OCR을 이용한 cost 검증

4-stage OCR pipeline

EasyOCR 정확도를 높이기 위한 전처리

Stage 1 · Crop

코스트 영역만 잘라낸다

```
roi = frame[y1:y2, x1:x2]
```

Stage 2 · HSV mask

파란 배경 제거 · 흰 숫자만 남김

```
cv2.inRange(hsv, lo, hi)
```

Stage 3 · Resize

3배 확대 · 작은 글자 인식을 향상

```
cv2.resize(mask, None, fx=3, fy=3)
```

Stage 4 · EasyOCR

숫자만 허용 · allowlist 강제

```
reader.readtext(img, allowlist='0123456789')
```

Sanity check · 게임 로직 기반 검증

OCR 결과를 그대로 PASS로 두지 않고 한번 더 검증

```
prev, curr = 1, ocr_result() # OCR이 '7' 반환
if abs(curr - prev) ≥ 5:      # 코스트는 1씩만 증가
    log.warning("OCR misread") # 룰 위반 → 무시
    curr = prev                # 이전 값 유지
```

슈트가 잡아낸 케이스

#11 시도 · OCR 오인식

expectedcost = 2

OCR rawcost = 7

delta|7-2| = 5 ≥ 5

actionreject + flag

잘못된 값을 그대로 받아 EX 스킬 미발동 →
정산 실패로 이어졌을 케이스. Sanity check가 사전 차단.



EasyOCR 을 이용한
START 발판 인식



보상 아이템 검증

3 classes, 3 responsibilities

결과 집계 — 3개 클래스의 협업

states/settlement_state.py

SettlementState

데미지 리포트 팝업을 OCR로 파싱해 저장하고, 다음 적이 남아 있는지에 따라 MapState로 복귀할지 RewardState로 갈지 결정합니다.

```
else: # 적 0명
    if self.context.enemy_count > 0:
        return BattleState(self.context) # 적이 먼저 공격한 케이스
    return RewardState(self.context) # 모든 적 처치 완료
```

데미지 리포트 팝업 + OCR 결과



states/reward_state.py

RewardState

보상 화면을 파싱하고 모든 검증 항목을 모아 test_summary.json 한 파일로 생성합니다. 전체 PASS/FAIL 판정도 여기서 결정됩니다.

```
result = "PASS" if all(
    c["ok"] for c in checks.values()
) else "FAIL"
```

test_summary.json 파일 내용

```
"EX_스킬_사용": {
  "status": "PASS",
  "detail": "총 15회 스킬 사용 기록",
  "skills": [
    {
      "student": "후우카",
      "slot": "SKILL_1",
      "cost": 5,
      "valid": true
    },
    {
      "student": "유우카",
      "slot": "SKILL_2",
      "cost": 3,
      "valid": true
    }
  ]
}
```

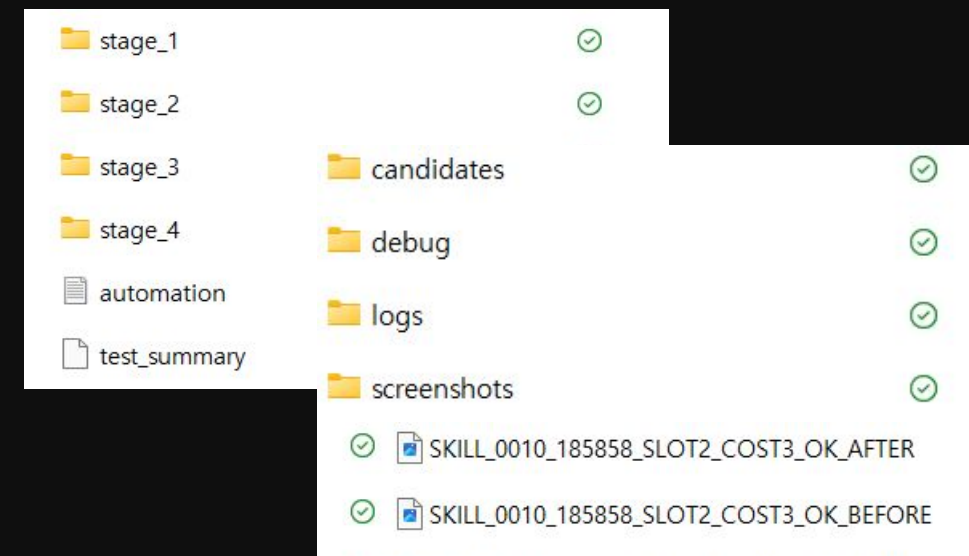
utils/file_manager.py

FileManager (Singleton)

로그·스크린샷·리포트 파일 핸들을 단일 인스턴스로 통제합니다. 여러 State가 동시에 같은 파일을 열어 발생하는 충돌을 차단합니다.

```
def __new__(cls, *args, **kwargs):
    if not cls._instance:
        cls._instance = super().__new__(cls) # 최초 1회만 생성
    return cls._instance
```

콘솔 로그 + 파일 트리



4 bugs found and fixed during development

개발 중 마주한 기술적 이슈와 해결

실제 발견 / 해결한 기술 이슈 (자체 디버깅)

BUG-001 · OCR misread	BUG-002 · Duplicate detect	BUG-003 · DPI mismatch
코스트 "1"이 "7"로 오인식 스킬 감지 오작동 발생 증상 · 한 프레임에 5 이상 급증으로 잘못 트리거 원인 · UI 폰트와 OCR 학습셋 차이 해결: 게임 메커니즘 활용 — 코스트는 자연 증가 시 1씩만 증가 → ≥5 변화는 OCR 오류로 간주	스킬 1회를 2~3회로 카운트 데미지 분석 오염 증상 · 동일 코스트 감소를 3프레임 연속 감지 원인 · 폴링 간격 33ms × OCR 지연 = race condition 해결: IDLE → COOLDOWN(0.7s) → IDLE 상태 머신 도입으로 중복 차단	DPI 스케일링 좌표 오차 고해상도 모니터에서만 발생 증상 · 125% · 150% DPI에서 클릭 좌표 어긋남 원인 · Windows 가상 좌표 vs 실제 픽셀 좌표 불일치 해결: SetProcessDPIAware() 호출로 실 픽셀 기준 좌표 통일

증거 패키지 · 재현 가능한 이슈 리포트 표준		
① 구조화 로그	② 스크린샷 파일명 컨벤션	③ test_summary.json
[03:16:02] [BATTLE] 스킬 사용 [후우카] SKILL_1 (소모: 5) ✓ [03:16:07] [BATTLE] 스킬 사용 [유우카] SKILL_2 (소모: 3) ✓ [03:16:09] [BATTLE] ⚠ 데미지 리포트 버튼 감지됨 [03:16:11] [BATTLE] 전투 종료 확정 → SettlementState 전환 [03:16:15] [DamageReport] 슬롯5: 유우카 (데미지: 5206)	SKILL_0001_031G02_SLOT1_COST5_OK_BEFORE.png ↑ 이벤트 타입 · 4자리 순번 ↑ 스테이지(31스테이지 2회차) ↑ 사용된 슬롯 1번 ↑ 소모된 코스트 5 ↑ 검증 결과(OK) ↑ 스킬 사용 직전 프레임	<pre>{ "name": "크레딧 포인트", "count": "600" }, { "name": "초급활동 보고서", "count": "1" }</pre> rewards_stage_4_190011

무엇을 · 얼마나 · 어떻게 측정했나

실행 결과 — 30회 무인 회귀 × 6개 검증 항목

시도 정의

1회 시도 = Normal 1-4 스테이지 무인 자동 실행 (발판 이동 → 전투 → 정산 → 보상) · 30회 반복 (#1 ~ #30) · 6개 검증 중 하나라도 실패하면 슈트가 스스로 FAIL 플래그

전체 통과율

96.7%

6/6 모두 통과 · 29 / 30 시도

목표

≥ 90%

시도당 소요 시간

-87%

수동 8분 → 자동 1분

자동화가 잡아낸 FAIL

1 건

#11 시도 · OCR 오인식 (코스트·EX스킬)

검증 항목별 통과율

어느 항목이 약했나 · 6개 검증 × 30회 시도

① 발판 이동		30 / 30
② 전투 진입		30 / 30
③ EX 스킬 사용		29 / 30
④ 코스트 소모		29 / 30
⑤ 대미지 기록		30 / 30
⑥ 보상 획득		30 / 30

시도당 8분 → 1분 단축 방법

① 사람 개입 제거 — 6항목을 동시에 자동 판정 · ② 대기 시간 단축 — 고정 sleep 대신 이벤트 기반 wait_until로 불필요한 대기 제거

Same methodology, different surface

게임 자동화 → Embedded SWQA 방법론 대응

🚩 게임 시스템 자동화 — 실습한 것	★ Embedded SWQA — 동일 원리가 적용됨
신호 획득 화면 캡처 → OpenCV 전처리 → EasyOCR 픽셀에서 값·상태를 추출	Embedded에서는 CAN 버스 · 센서 값 · ECU 메모리 dump CANoe · dSPACE로 신호를 취득 — 표준화된 채널
시나리오 구조 FSM — Init → Map → Battle → Settle → Reward 상태 머신으로 자동화 흐름 모델링	Embedded에서는 AUTOSAR SW-C 상태 · ECU 부팅 시퀀스 동일한 상태 머신 모델 — 각 상태별 전이 조건을 TC로 표현
이상값 판정 OCR 결과 sanity check ($\Delta cost \geq 5 \rightarrow reject$) 게임 로직 기반 범위 검증	Embedded에서는 센서 스파이크 필터 · 범위 밖 값 rejection 차량 속도 -50km/h 같은 물리적으로 불가능한 값을 차단
반복 회귀 30회 무인 회귀 · test_summary.json 동일 시나리오 반복 → 재현 가능성 확보	Embedded에서는 HIL 회귀 테스트 · CANoe test module 반복 실행 같은 접근 — 로그 형식만 vTESTstudio 리포트로 대체

결론 · 한 줄

게임 UI는 표준 신호가 없어 픽셀 기반으로 상태를 추정해야 했습니다. Embedded 환경에서는 CAN 메시지 · 센서 값이 정형화된 신호로 제공되므로, 동일한 검증 로직을 더 정밀하게 적용할 수 있습니다.

Summary

감사합니다.

가져갈 것 · 01

검증 사고법

신호를 받아 → 범위 검사 → 상태 전이 판정 → 증거 기록. 이 사이클은 U든 CAN이든 동일하게 적용됩니다.

가져갈 것 · 02

재현 가능한 증거

log + screenshot + JSON 세 개를 항상 함께 남기는 컨벤션. FAIL이 생기면 5분 안에 재현하고 개발자에게 넘길 수 있도록.

가져갈 것 · 03

도구 이식

OpenCV·OCR·FSM을 독학으로 작동시킨 경험을 CANoe, AUTOSAR 같은 embedded 표준 도구 학습에도 반복 적용하겠습니다.

지원자

이시영

Embedded SWQA 지원 · 2026

연락처

syoun8296@gmail.com